

Modelo de Claves Booleanas



Consigna 2:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Asegurar las descargas necesarias de los recursos de NLTK
nltk.download('punkt')
nltk.download('stopwords')

# 1. Base de datos de documentos suministrada (Corpus de Civilizaciones Antiguas)
documentos = {
    "doc1": "Los egipcios construyeron las pirámides y desarrollaron una escritura jeroglífica.",
    "doc2": "La civilización romana fue una de las más influyentes en la historia occidental.",
    "doc3": "Los mayas eran expertos astrónomos y tenían un avanzado sistema de escritura.",
    "doc4": "La antigua Grecia sentó las bases de la democracia y la filosofía moderna.",
    "doc5": "Los sumerios inventaron la escritura cuneiforme y fundaron las primeras ciudades."
}

# Obtener el conjunto oficial de palabras vacías (stop words) en español
stop_words = set(stopwords.words('spanish'))

# 2. Función de preprocesamiento: Tokenización y Limpieza
def limpiar_texto(texto):
    # Tokenización y conversión a minúsculas
    tokens = word_tokenize(texto.lower())
    # Filtrado eliminando stop words, signos de puntuación y caracteres especiales
    tokens_limpios = [t for t in tokens if t.isalnum() and t not in stop_words]
    return tokens_limpios

# 3. Construcción del Índice Invertido
indice_invertido = {}
for doc_id, texto in documentos.items():
    palabras_clave = limpiar_texto(texto)
    for palabra in palabras_clave:
        if palabra not in indice_invertido:
            indice_invertido[palabra] = set()
        indice_invertido[palabra].add(doc_id)

# 4. Motor de Procesamiento de Consultas Booleanas
def buscar_booleano(consulta):
    partes = consulta.split()
```

```
# Manejo de consultas compuestas con estructura: [Término1] [OPERADOR] [Término2]
if len(partes) == 3:
    t1, operador, t2 = partes[0].lower(), partes[1].upper(), partes[2].lower()
```

```
# Obtención de los conjuntos de documentos asociados (set vacío por defecto)
set1 = indice_invertido.get(t1, set())
set2 = indice_invertido.get(t2, set())
```

```
if operador == "AND":
    return set1.intersection(set2)
elif operador == "OR":
    return set1.union(set2)
elif operador == "NOT":
    return set1.difference(set2)
```

```
# Manejo de consultas de término único
elif len(partes) == 1:
    return indice_invertido.get(partes[0].lower(), set())
```

```
return set()
```

```
# 5. Interfaz de ejecución y pruebas automáticas para la Consigna 2
```

```
if __name__ == "__main__":
    print("=====")
    print("SISTEMA DE RECUPERACIÓN BOOLEANA - CONSIGNA 2")
    print("=====")
```

```
# Casos de prueba requeridos en la consigna 2
```

```
pruebas = [
    "egipcios AND pirámides",
    "escritura OR astrónomos"
]
```

```
for consulta in pruebas:
    resultados = buscar_booleano(consulta)
    print(f"Consulta: '{consulta}'")
    print(f" ♦ Documentos encontrados: {resultados}\n")
```

Validación de Resultados (Output del Programa):

Al ejecutar este script específico para el segundo corpus, el sistema procesa los conjuntos lógicos y devuelve los siguientes resultados exactos:

1. Consulta: egipcios AND pirámides

- **Resultado:** {'doc1'}
- *Explicación:* Ambos términos aparecen en el doc1 ("Los **egipcios** construyeron las **pirámides**..."). La intersección lógica da como resultado único este documento.

2. Consulta: escritura OR astrónomos

- **Resultado:** {'doc1', 'doc3', 'doc5'}
- *Explicación:* El término "escritura" está presente en el doc1 (jeroglífica), doc3 (sistema de escritura) y doc5 (cuneiforme). Por otro lado, "astrónomos" se encuentra en el doc3. Al aplicar la unión lógica (OR), se consolidan los documentos válidos sin duplicados.